TECHNICAL UNIVERSITY OF MUNICH
SCHOOL OF COMPUTATION, INFORMATION, AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE, SS 2026

Operations Research

Prof. Dr. Martin Bichler,
Halil Bayrak, Janik Bürgermeister, Teodora Dobos,
Jan Höhener, Abheek Ghosh

Solution to the Training Exercises, Sheet 9

Training Exercises

Exercise T9. 1. *Euler vs Hamilton*

Problem

For each of the tasks give a graph G with at least three vertices which fulfil the following properties:

a) G is both Eulerian and Hamiltonian.

b) G is neither Eulerian nor Hamiltonian.

c) G is not Eulerian but Hamiltonian.

d) G is Eulerian but not Hamiltonian.

e) G does not represent a symmetric TSP instance.

f) G represents a symmetric TSP instance that is not metric.

Solution

Consider the examples as shown below

a)
- Eulerian: Yes, since all vertices have even degree.
- Hamiltonian: Yes, traversing the edges in any order gives a Hamilton cycle.

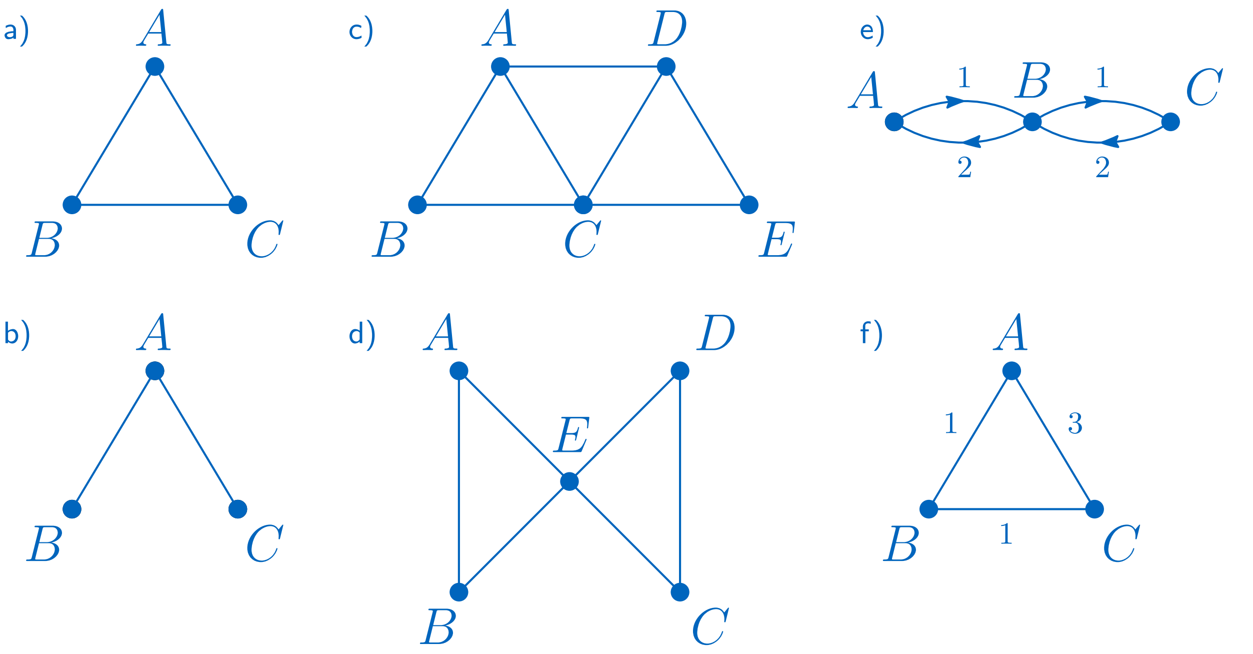
b)
- Eulerian: No, B and C have odd degree.
- Hamiltonian: No, the graph is a tree and has thus no cycle.

c)
- Eulerian: No, A and D have odd degree.
- Hamiltonian: Yes, $ADECBA$ is one of the possible cycles.

d)
- Eulerian: Yes, since all vertices have even degree.
- Hamiltonian: No, in any cycle connecting all five vertices, E must be visited multiple times.

e) The edge from A to B does not have the same length as the edge from B to A.

f) A metric TSP requires two conditions: edge symmetry and the triangle inequality. While the graph is symmetric due to being undirected, the inequality $AB + BC \geq AC$ does not hold, violating the metric condition.



Exercise T9. 2. *Filling of ATMs*

Problem

The bank *Total Untrustworthy Money (TUM)* maintains a number of ATMs for its retail customers at locations A, B, C, D, E, F, G around its headquarters in Garching-Hochbrück (H). Each of the ATMs must be refilled every seven days. For this purpose, the bank sends two employees from H with a small cash transporter to drive off the ATMs of the respective day and change the cash cassettes. The cost of a trip depends linearly on the distance traveled and the distances between the locations are known. Since the employees in question also have other tasks that need to be completed, they are only supposed to go out to fill the ATMs twice a week at the most. Also, no ATM can be visited multiple times within one week. The bank is now asking for a cost-minimizing solution.

a) To which of the problem classes mentioned in the lecture within integer optimization (Assignment, Knapsack, Bin Packing, Set Covering, Traveling Salesperson) can the described situation be assigned? Give reasons for your answer.

b) Model the described problem as a (mixed) integer linear program. If the number of necessary constraints is very large, describe them compactly.

c) What changes for the model and program if employees are only asked to leave for a trip once a week?

Solution

a) This is a vehicle routing problem. From a central depot, places have to be reached on round trips. Note that planning by days of the week is superfluous for the task. One can limit oneself only to the tours.

b) Let $N = \{A, B, C, D, E, F, G, H\}$ and d_{ij} be the known distances between i and j with $i, j \in N$. For simplicity, we assume $d_{ii} = 0$ for all i.

Variables

x_{ij} binary, Edge from i to j is used for a Tour, set all $x_{ii} = 0$

Objective

See TSP from lecture:

$$\min z = \sum_{i \in N} \sum_{j \in N} d_{ij} x_{ij}$$

Constraints

See TSP from lecture:

$$\sum_{i \in N} x_{ij} = 1 \text{ for all } j \in N \setminus \{H\}$$
$$\sum_{j \in N} x_{ij} = 1 \text{ for all } i \in N \setminus \{H\}$$
$$x_{ij} \in \{0, 1\} \text{ for all } i, j \in N$$

Only node H must be considered separately, because several tours can start and end here. There must be only the same number

$$\sum_{i \in N \setminus \{H\}} x_{iH} - \sum_{j \in N \setminus \{H\}} x_{Hj} = 0 \tag{1}$$

and not more than 2

$$\sum_{i \in N \setminus \{H\}} x_{iH} \leq 2. \tag{2}$$

In the lecture we discussed the prohibition of short cycles. This is now similar, since we do not want subcycles in which H does not occur.

$$\sum_{\{i,j\} \subseteq U^{-H}} x_{ij} \leq |U^{-H}| - 1 \text{ für alle } U^{-H} \subseteq N \setminus H \text{ mit } 2 < |U^{-H}| < |N| - 1 \tag{3}$$

c) It means that the total number of tours is now bounded by 1 and it is a Travelling Salesperson problem whose modeling has already been discussed in lecture. Instead of (1), (2), and (3) we now get

$$\sum_{i \in N \setminus \{H\}} x_{iH} = 1$$
$$\sum_{j \in N \setminus \{H\}} x_{Hj} = 1$$

and

$$\sum_{\{i,j\}\subseteq U} x_{ij} \leq |U| - 1 \text{ for all } U \subseteq N \text{ with } 2 < |U| < |N| - 1$$

Self-study Exercises

Exercise S9. 1. *Two Travelers*

Problem

You intend to establish a delivery service in Garching. You have two delivery vans at your disposal for this purpose. You want to deliver to your customers in such a way that you incur minimal transportation costs. Given an undirected complete graph $G = (V, E)$ with cost function $c : E \to \mathbb{R}_{\geq 0}$, where each node represents one customer and $c_{ij} := c(\{i, j\})$ models the travel cost from customer i to customer j. Your warehouse is located at node $1 \in V$. This is where the tours of the delivery trucks start and end. Of course, each customer must be on the tour of one of your delivery trucks.

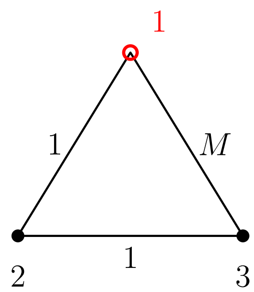
a) Give a graph with at least 3 nodes and a cost function such that it is cheaper to use only one of the delivery carts.

b) Give a graph and a cost function such that it is optimal to visit a node (except 1) more than once. Does your solution satisfy the triangle inequality?

c) You now additionally require that the cases in (a) and (b) must not occur, i.e., each delivery truck must make a true round trip over a subset of the nodes and each node except 1 must be visited exactly once in total. Formulate an IP to solve the problem.

Solution

a) und (b). Consider the following graph:



where the edges are labeled with their weights, and M is large ($M > 2$ is already sufficient here). Then it cannot be optimal to use the edge $\{1, 3\}$. So the optimal route is $1 \to 2 \to 3 \to 2 \to 1$. Here, only one of the two delivery vans is used and node 2 is visited twice. The weight function violates the triangle inequality: going from 1 directly to 3 is more expensive than taking the detour via 2.

c) For $i, j \in V$ with $i \neq j$ and $k \in \{1, 2\}$, introduce variables $x_{ij}^k \in \{0, 1\}$ that describe whether van k uses the route segment from i to j. First, each node except 1 must be visited exactly once:

$$\sum_{k=1}^{2} \sum_{i \neq j} x_{ij}^k = 1 \, \forall j \in V \setminus \{1\}$$

Furthermore, each node (even 1) must be visited by a delivery truck exactly as many times as it is left:

$$\sum_{i \neq j} x_{ij}^k = \sum_{i \neq j} x_{ji}^k \, \forall j \in V \, \forall k \in \{1, 2\}$$

We now have to prevent - similar to the TSP - that no subtours are created which do not contain 1. This can be done, for example, with the SEC constraints:

$$\sum_{i,j\in U,\, i\neq j} x_{ij}^k \leq |U| - 1 \,\forall U \subseteq V \setminus \{1\} \,\forall k \in \{1,2\}$$

Finally, both delivery trucks are supposed to drive real tours

$$\sum_{i,j\in V,\, i\neq j} x_{ij}^k \geq 1 \,\forall k \in \{1,2\}$$

and 1 may not be visited during the tours, so is visited exactly twice in total:

$$\sum_{k=1}^{2} \sum_{i\neq 1} x_{i1}^k = 2$$

They want to minimize their costs, so they want to solve the following objective:

$$\min \sum_{i,j\in V,\, i\neq j} c_{ij}(x_{ij}^1 + x_{ij}^2).$$

Alternative solution for constraints 4 and 5: The constraints

$$\sum_{i,j\in V,\, i\neq j} x_{ij}^k \geq 1 \,\forall k \in \{1,2\} \text{ und}$$

$$\sum_{k=1}^{2} \sum_{i\neq 1} x_{i1}^k = 2$$

can be replaced by

$$\sum_{j\neq 1} x_{j1}^k = 1 \,\forall k \in \{1,2\}.$$

Each van uses an edge to 1 exactly once, so visits this node exactly once, and must also make a real trip, otherwise it could not return to 1.

Exercise S9. 2. *Slitherlink*

Problem

In the logic puzzle Slitherlink, an $n \times m$ point grid is given. Two points are adjacent if they are directly next to or above/below each other in the grid. The goal is to connect adjacent points with edges, under the following conditions:

- Diagonal edges are not allowed. Only points that are directly next to or above/below each other can be connected.
- At the end, a single closed loop should be formed.
- The grid contains additional 'constraint numbers' between 0 and 3, which are located between 4 grid points. These indicate how many edges must exist between these 4 points.

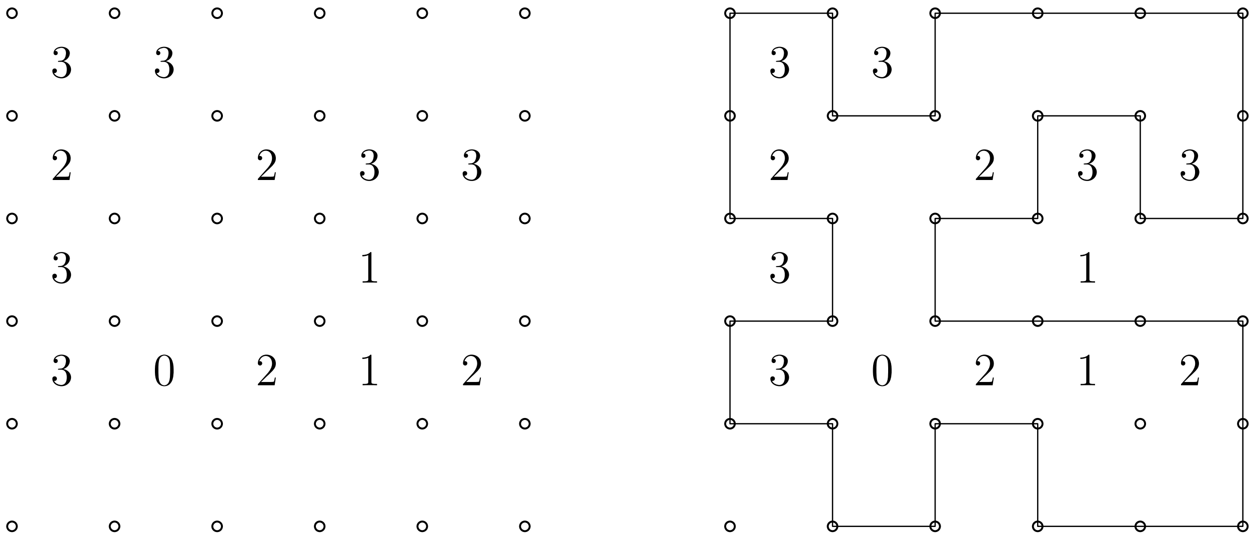


Figure 1: Slitherlink puzzle with a corresponding solution

Consider the following example and a corresponding solution.

a) Model Slitherlink as an ILP, but initially leave out the subtour elimination constraints. Why can't you use the standard condition from the lecture?

b) The following alternative for modeling subtour elimination constraints with real-valued variables $u_i \geq 0$ for the classic TSP goes back to Miller, Tucker, and Zemlin:

$$\begin{aligned} u_1 &= 1 && \\ u_i &\geq 2 && \text{for } i \in V \setminus \{1\} \\ u_i &\leq n && \text{for } i \in V \\ u_i - u_j + 1 &\leq (n-1)(1 - x_{ij}) && \text{für } i, j \in V, j \neq 1 \end{aligned}$$

Explain how these constraints eliminate subtours. Under what conditions can you use these constraints in Slitherlink as well?

Solution

a) We use the indices $i = 1, \ldots, n$ for the n rows and the indices $j = 1, \ldots, m$ for the m columns. For each constraint number b_k with $k \in K$, we define the function $L(k)$, which gives us the top-left grid point adjacent to b_k.

Additionally, we employ the following trick and add 'virtual' points $(0, j)$, $(n+1, j)$ for $j = 1, \ldots, m$ and $(i, 0)$, $(i, m+1)$ for $i = 1, \ldots, n$. This allows for a more compact notation of the required variables and the IP.

Variables

$h_{i,j} \in \{0,1\}$ for $i = 1, \ldots, n, j = 0, \ldots, m$	add horizontal edge from point (i, j) to point $(i, j+1)$
$v_{i,j} \in \{0,1\}$ for $i = 0, \ldots, n, j = 1, \ldots, m$	add vertical edge from point (i, j) to point $(i+1, j)$
$x_{i,j} \in \{0,1\}$ for $i = 1, \ldots, n, j = 1, \ldots, m$	point (i, j) is part of the tour

Constraints

$$
\begin{aligned}
&h_{i,j-1} + h_{i,j} + v_{i-1,j} + v_{i,j} && = 2x_{i,j} && \text{for } i = 1,\dots,n, j = 1,\dots,m && (1)\\
&h_{i,j} + h_{i+1,j} + v_{i,j} + v_{i,j+1} && = b_k && \text{for } k \in K, (i,j) = L(k) && (2)\\
&h_{i,j} && = 0 && \text{for } i = 1,\dots,n; j = 0, m && (3)\\
&v_{i,j} && = 0 && \text{for } i = 0, n; j = 1,\dots,m && (4)\\
&h_{ij} && \in \{0,1\} && \text{for } i = 1,\dots,n, j = 0,\dots,m && (5)\\
&v_{ij} && \in \{0,1\} && \text{for } i = 0,\dots,n, j = 1,\dots,m && (6)\\
&x_{ij} && \in \{0,1\} && \text{for } i = 0,\dots,n, j = 1,\dots,m && (7)
\end{aligned}
$$

The subtour elimination constraints from the lecture cannot be used because not every point necessarily needs to be visited, and we do not know how many points need to be visited. Therefore, it is unclear what size tours are still permissible and which ones we need to forbid.

b) Each node is assigned a label in the form of a number from 1 to n. For example, node 1 gets the label 1. If a node i is connected to another node j by the edge (i, j) in the tour, the successor node j must receive a label that is at least 1 greater than i. Only if the successor node is node 1 can the label of 1 be lower than that of its predecessor node. Thus, the order in a tour is defined, and only tours that include node 1 are allowed, as otherwise, the labels would have to continuously increase, which is not possible in a loop. However, due to the limitation of the edges adjacent to 1, there can only be exactly one tour that includes 1.

We can use this methodology in our problem if we identify a point that is certainly in the tour. This is easily possible, for example, if we find a constraint number 3 (then every adjacent point is in the tour). One of these points then receives the label 1.